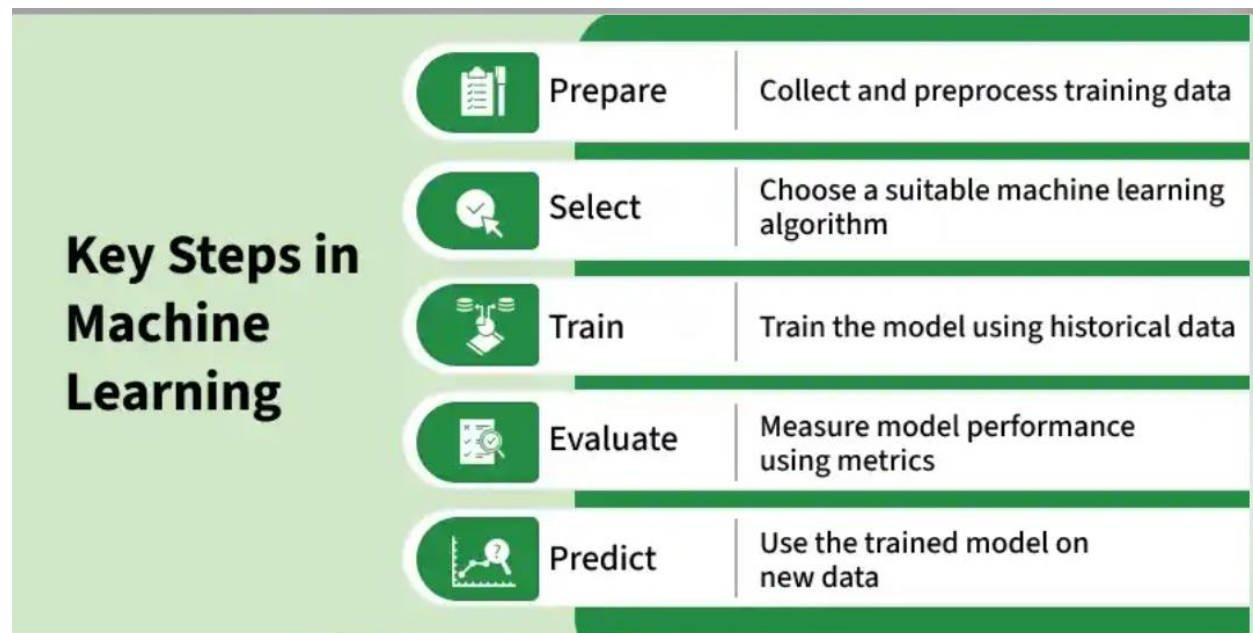


Unit – IV

Machine Learning with Python Tutorial

Machine Learning with Python focuses on building systems that can learn from data and make predictions or decisions without being explicitly programmed. Python provides simple syntax and useful libraries that make machine learning easy to understand and implement, even for beginners.

- Preparing data for training machine learning models.
- Selecting suitable algorithms for a problem.
- Training models using historical data.
- Evaluating model performance.
- Making predictions on new data.



Why Python for Machine Learning?



Strong Data Handling Support

We use Pandas and NumPy to load, clean, and prepare data



Machine Learning Algorithm

We use Scikit-learn for model making.



Deep Learning Support

We use TensorFlow, Keras, and PyTorch to build neural networks.



Model Evaluation & Tuning

We use Scikit-learn for evaluation and hyperparameter tuning.

Machine Learning Use Cases



Image Recognition



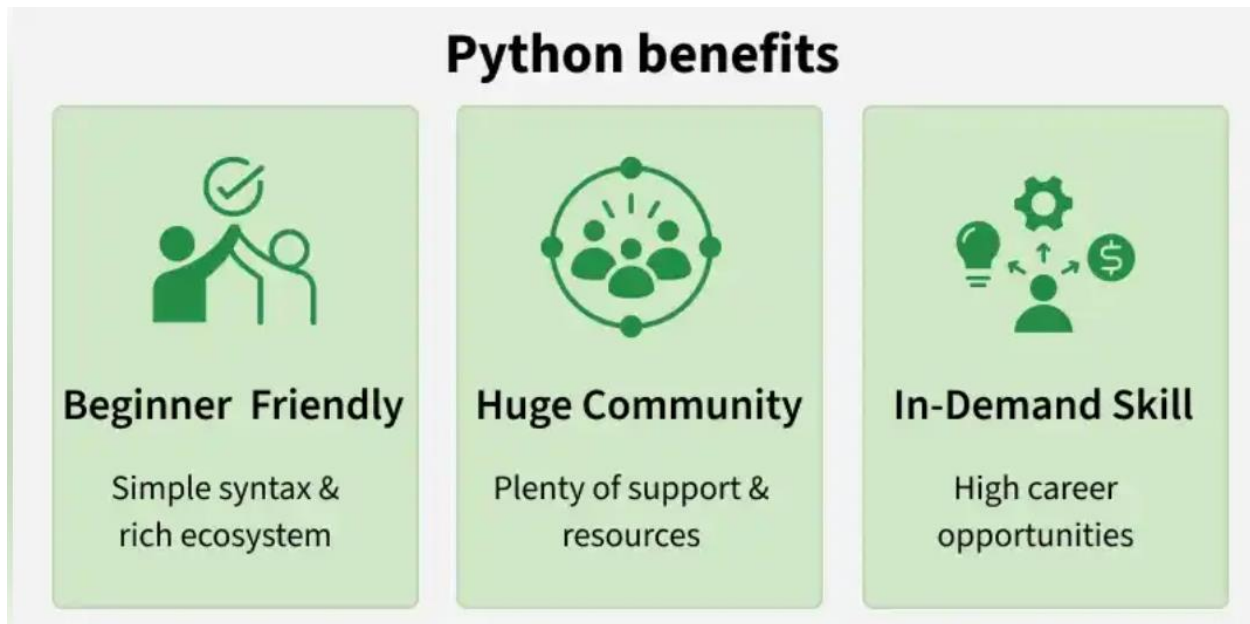
Natural Language Processing (NLP)



Recommendation Systems



Predictive Maintenance



Getting Started with Machine Learning

Before starting this tutorial, it is important to understand the core ideas behind machine learning and how it differs from traditional programming.

- [Introduction to Machine Learning](#)
- [Types of Machine Learning](#)
- [Machine Learning vs Deep Learning](#)
- [Machine Learning vs Data Science](#)
- [Applications of Machine Learning](#)

Basic Python Concepts

Python is the most widely used language for machine learning. A strong foundation in Python is required to understand model implementation and workflows.

- [Download and Install Python 3](#)
- [Input and Output in Python](#)
- [Python Variables](#)
- [Python Keywords](#)
- [Python Data Types](#)

- [Python Operators](#)
- [Conditional Statements in Python](#)
- [Loops in Python](#)
- [Python Functions](#)
- [Python String](#)
- [Python Lists](#)
- [Python Dictionary](#)
- [Python Tuples](#)
- [Sets in Python](#)
- [Python Exception Handling](#)

Python Libraries for Machine Learning

To work efficiently with machine learning, you should be familiar with the following libraries:

- [NumPy for Numerical Computing](#)
- [Pandas for Data Manipulation](#)
- [Matplotlib for Data Visualization](#)
- [Seaborn for Data Visualization](#)
- [Scikit-learn for Machine Learning](#)

Data Preparation for Machine Learning

Data preparation is a critical step where raw data is cleaned and transformed to make it suitable for model training.

- [Loading Dataset using Pandas](#)
- [Understanding the Dataset Structure](#)
- [Data Preprocessing](#)
- [Data Cleansing](#)
- [Handling Missing Values](#)
- [Removing Duplicate Records](#)

- [Feature Scaling using StandardScaler and MinMaxScaler](#)
- [Encoding Categorical Data](#)
- [Train-Test Split using Scikit-learn](#)

Supervised Learning

Supervised learning uses labeled data to train models that can predict outputs for unseen data.

Regression Algorithms

Regression algorithms are used to predict continuous numerical values.

- [Introduction to Regression](#)
- [Simple Linear Regression](#)
- [Multiple Linear Regression](#)
- [Polynomial Regression](#)
- [Ridge Regression](#)
- [Lasso Regression](#)
- [Elastic Net Regression](#)
- [K-Nearest Neighbors Regression](#)
- [Support Vector Regression \(SVR\)](#)
- [Decision Tree Regression](#)
- [Random Forest Regression](#)

Classification Algorithms

Classification algorithms are used to predict discrete class labels by learning patterns from labeled data.

- [Introduction to Classification](#)
- [Logistic Regression](#)
- [K-Nearest Neighbors \(KNN\)](#)

- Support Vector Machine (SVM)
- Naive Bayes Algorithm
- Decision Tree Classifier
- Random Forest Classifier

Unsupervised Learning

Unsupervised learning works with unlabeled data to discover hidden patterns and structures.

Clustering Algorithms

Clustering algorithms group similar data points together based on their features without using labeled data.

- K-Means Clustering
- K-means++ Clustering
- Fuzzy Clustering
- Hierarchical Clustering
- DBSCAN Algorithm
- Mean Shift Clustering
- Gaussian Mixture Models
- Dirichlet Process Mixture Models
- DBSCAN Clustering
- OPTICS Clustering

Association Rule Learning

Association rule learning is used to discover relationships and frequent patterns among variables in large datasets.

- Apriori Algorithm
- FP-Growth Algorithm

- Eclat Algorithm

Dimensionality Reduction Techniques

Dimensionality reduction techniques reduce the number of features while preserving important information in the data.

- Principal Component Analysis (PCA)
- Feature Importance
- Linear Discriminant Analysis (LDA)
- t-SNE (t-Distributed Stochastic Neighbor Embedding)
- Independent Component Analysis (ICA)
- Locally Linear Embedding

Reinforcement Learning

Reinforcement learning interacts with environment and learn from them based on rewards.

Model-Based Methods

These methods use a model of the environment to predict outcomes and help the agent plan actions by simulating potential results.

- Markov decision processes (MDPs)
- Bellman equation
- Value iteration algorithm
- Monte Carlo Tree Search

Model-Free Methods

The agent learns directly from experience by interacting with the environment and adjusting its actions based on feedback.

- Q-Learning
- Deep Q-Learning

- SARSA
- Monte Carlo Methods
- Reinforce Algorithm
- Actor-Critic Algorithm
- Asynchronous Advantage Actor-Critic (A3C)

Ensemble Learning

Ensemble learning combines multiple models to improve prediction accuracy.

- Introduction to Ensemble Learning
- Bagging Algorithm
- Boosting Algorithm
- AdaBoost
- Gradient Boosting
- XGBoost
- CatBoost
- LightGBM
- Stacking

Forecasting Models

Forecasting models are used to predict future values based on historical time-series data and observed trends.

- ARIMA (Auto-Regressive Integrated Moving Average)
- SARIMA (Seasonal ARIMA)
- Exponential Smoothing
- Exponential Smoothing (Holt-Winters)

Model Evaluation and Validation

Model evaluation helps in measuring how well a machine learning model performs.

- Confusion Matrix
- Precision, Recall and F1-Score

- ROC Curve and AUC
- Cross Validation
- Bias and Variance
- Overfitting and Underfitting
- Regularization

Python Arrays

Lists in Python are the most flexible and commonly used data structure for sequential storage. They are similar to arrays in other languages but with several key differences:

- **Dynamic Typing:** Python lists can hold elements of different types in the same list. We can have an integer, a string and even other lists all stored within a single list.
- **Dynamic Resizing:** Lists are dynamically resized, meaning you can add or remove elements without declaring the size of the list upfront.
- **Built-in Methods:** Python lists come with numerous built-in methods that allow for easy manipulation of the elements within them, including methods for appending, removing, sorting and reversing elements.

```
a = [1, "Hello", [3.14, "world"]]
a.append(2) # Add an integer to the end
print(a)
```

Output

```
[1, 'Hello', [3.14, 'world'], 2]
```

NumPy Arrays

NumPy arrays are a part of the NumPy library, which is a powerful tool for numerical computing in Python. These arrays are designed for high-performance operations on large volumes of data and support multi-dimensional arrays and matrices. This makes them ideal for complex mathematical computations and large-scale data processing.

Features:

- **Multi-dimensional support:** NumPy arrays can handle more than one dimension, making them suitable for matrix operations and more complex mathematical constructs.
- **Broad broadcasting capabilities:** They can perform operations between arrays of different sizes and shapes, a feature known as broadcasting.
- **Efficient storage and processing:** NumPy arrays are stored more efficiently than Python lists and provide optimized performance for numerical operations.

```
import numpy as np
```

```
a = np.array([1, 2, 3, 4])
```

```
# Element-wise operations
```

```
print(a * 2)
```

```
# Multi-dimensional array
```

```
res = np.array([[1, 2], [3, 4]])
```

```
print(res * 2)
```

Output

```
[2 4 6 8]
```

```
[[2 4]
```

```
[6 8]]
```

Python Arrays

In Python, array is a collection of items stored at contiguous memory locations. The idea is to store multiple items of the same type together. Unlike Python lists (can store elements of mixed types), arrays must have all elements of same type. Having only homogeneous elements makes it memory-efficient. It requires a Typecode during initialization to define the C-type.

```
import array as arr
```

```
a = arr.array('i', [1, 2, 3])
```

```
# accessing First Array
```

```
print(a[0])
```

```
# adding element to array
```

```
a.append(5)
```

```
print(a)
```

Output

```
1  
array('i', [1, 2, 3, 5])
```

Explanation: parameter '**i**' is the typecode, it tells Python to treat the elements as signed integers ([1, 2, 3]) of a specific byte size (usually 2 or 4 bytes).

Create an Array in Python

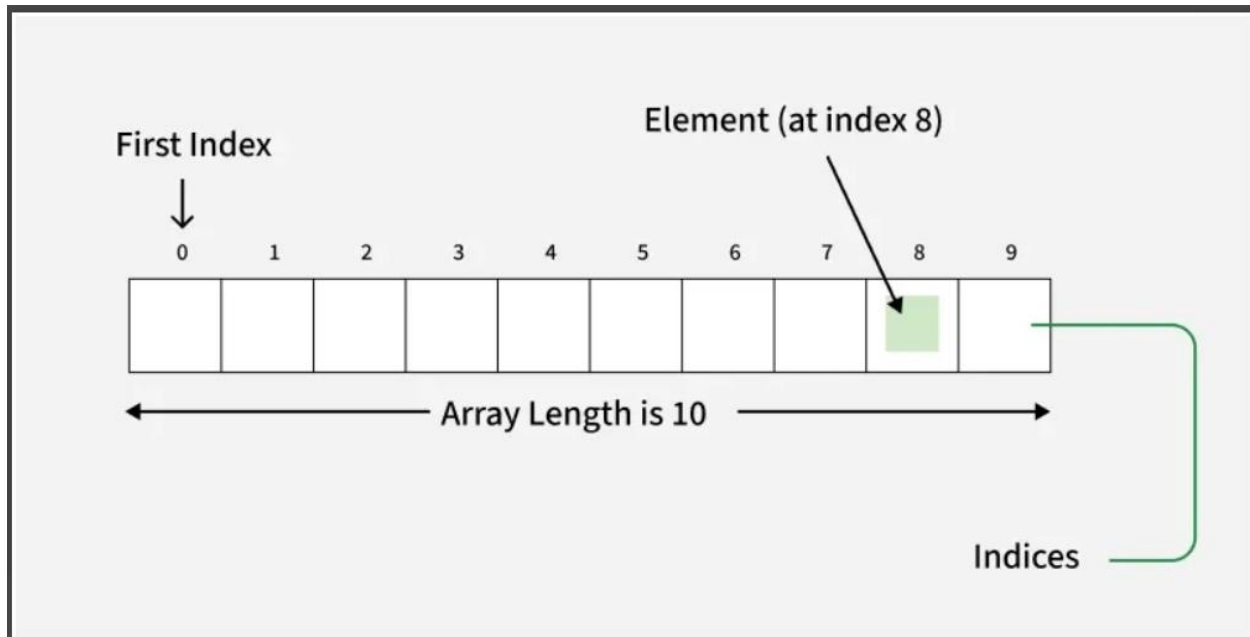
Array in Python can be created by importing an array module. **array(*data_type* , *value_list*)** is used to create array in Python with data type and value list specified in its arguments.

```
import array as arr  
a = arr.array('i', [1, 2, 3])
```

```
for i in range(0, 3):  
    print(a[i], end=" ")
```

Output

```
1 2 3
```



Adding Elements to an Array

Elements can be added to the Python Array by using built-in [insert\(\)](#) function. Insert is used to insert one or more data elements into an array. Based on the requirement, a new element can be added at the beginning, end, or any given index of array. [append\(\)](#) is also used to add the value mentioned in its arguments at the end of the Python array.

```
import array as arr
a = arr.array('i', [1, 2, 3])
print(*a)

a.insert(1, 4) # Insert 4 at index 1
print(*a)
```

Output

```
1 2 3
1 4 2 3
```

Accessing Array Items

In order to access the array items refer to the index number. Use the index operator [] to access an item in a array in Python. The index must be an integer.

Removing Elements from the Array

Elements can be removed from the Python array by using built-in remove() function. It will raise an Error if element doesn't exist. Remove() method only removes the first occurrence of the searched element. To remove range of elements, we can use an iterator.

pop() function can also be used to remove and return an element from the array. By default it removes only the last element of the array. To remove element from a specific position, index of that item is passed as an argument to pop() method.

```
import array
a = array.array('i', [1, 2, 3, 1, 5])
```

```
# remove first occurrence of 1
```

```
a.remove(1)
```

```
print(a)
```

```
# remove item at index 2
```

```
a.pop(2)
```

```
print(a)
```

Output

```
array('i', [2, 3, 1, 5])
```

```
array('i', [2, 3, 5])
```

```
import array as arr
```

```
a = arr.array('i', [1, 2, 3, 4, 5, 6])
```

```
print(a[0])
```

```
print(a[3])
```

```
b = arr.array('d', [2.5, 3.2, 3.3])
```

```
print(b[1])
```

```
print(b[2])
```

Output

```
1
```

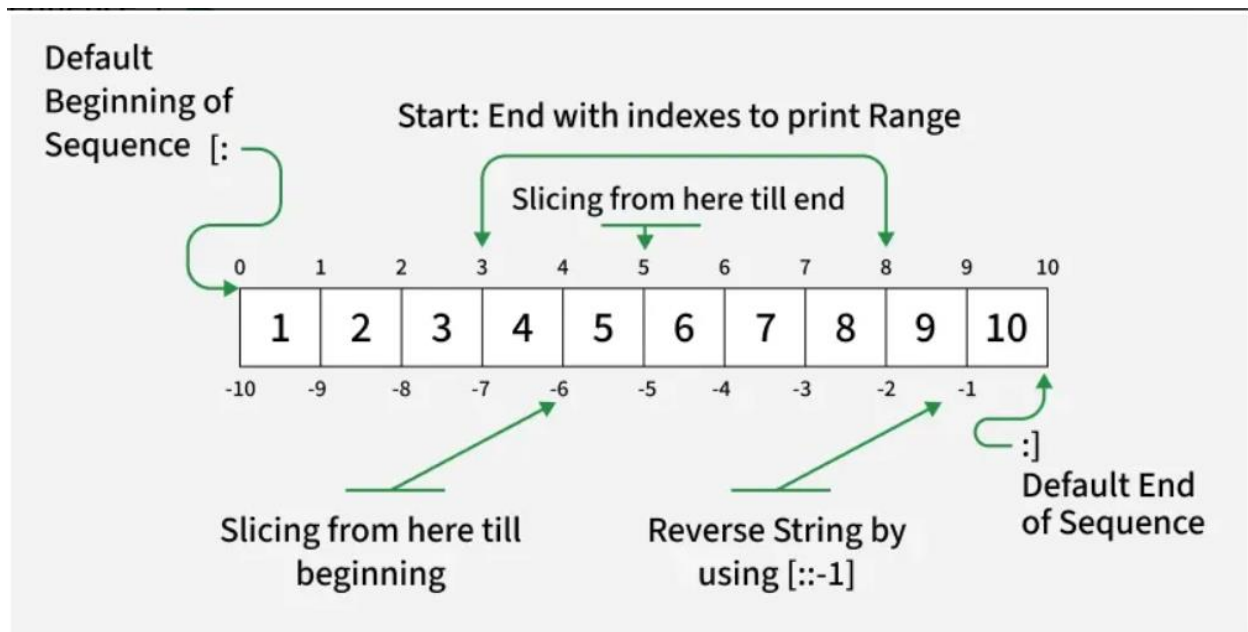
```
4
```

```
3.2
```

```
3.3
```

Slicing of an Array

In Python array, there are multiple ways to print the whole array with all the elements, but to print a specific range of elements from the array, we use Slice operation.



- Elements from beginning to a range use `[:Index]`
- Elements from end use `[:-Index]`
- Elements from specific Index till the end use `[Index:]`
- Elements within a range, use `[Start Index:End Index]`
- Print complete List, use `[:]`.
- For Reverse list, use `[::-1]`.

```
import array as arr
```

```
a = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
b = arr.array('i', a)
```

```
res = a[3:8]
```

```
print(res)
```

```
res = a[5:]
```

```
print(res)
```

```
res = a[:]  
print(res)
```

Output

```
[4, 5, 6, 7, 8]  
[6, 7, 8, 9, 10]  
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Searching Element in an Array

In order to search an element in the array we use a python in-built index() method. This function returns the index of the first occurrence of value mentioned in arguments.

```
import array  
a = array.array('i', [1, 2, 3, 1, 2, 5])  
# index of 1st occurrence of 2  
print(a.index(2))  
# index of 1st occurrence of 1  
print(a.index(1))
```

Output

```
1  
0
```

Updating Elements in an Array

In order to update an element in the array we simply reassign a new value to the desired index we want to update.

```
import array
a = array.array('i', [1, 2, 3, 1, 2, 5])
# update item at index 2
a[2] = 6
print(a)
# update item at index 4
a[4] = 8
print(a)
```

Output

```
array('i', [1, 2, 6, 1, 2, 5])
array('i', [1, 2, 6, 1, 8, 5])
```

Different Operations on Python Arrays

Counting Elements in an Array

We can use `count()` method to count given item in array.

```
import array
a = array.array('i', [1, 2, 3, 4, 2, 5, 2])
count = a.count(2)
print(count)
```

Output

```
3
```

Reversing Elements in an Array

In order to reverse elements of an array we need to simply use reverse method.

```
import array
a = array.array('i', [1, 2, 3, 4, 5])
a.reverse()
print(*a)
```

Output

```
5 4 3 2 1
```

Extend Element from Array

In Python, an array is used to store multiple values or elements of the same datatype in a single variable. The **extend()** function is simply used to attach an item from iterable to the end of the array. In simpler terms, this method is used to add an array of values to the end of a given or existing array.

```
list.extend(iterable)
```

Here, all the element of iterable are added to the end of a.

```
import array as arr
a = arr.array('i', [1, 2, 3,4,5])
a.extend([6,7,8,9,10])
print(a)
```

Output

```
array('i', [1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
```

Introduction to NumPy-Array Creation in NumPy

What is NumPy?

NumPy (Numerical Python) is a powerful Python library used for:

- Working with arrays
- Performing mathematical operations
- Handling matrices
- Scientific and numerical computations

Why Use np?

It makes code:

- Shorter
- Cleaner
- Easier to write and read

NumPy - Array Creation

NumPy Arrays are grid-like structures similar to lists in Python but optimized for numerical operations. The most straightforward way to create a NumPy array is by converting a regular Python list into an array using the `np.array()` function.

```
import numpy as np
# One-dimensional array
arr1 = np.array([1, 2, 3, 4, 5])
print(arr1)
```

```
# Two-dimensional array
arr2 = np.array([[1, 2], [3, 4]])
print(arr2)
```

Output

```
[1 2 3 4 5]

[[1 2]

 [3 4]]
```

Creating Arrays with Specific Values

For assigning a specific values. NumPy provides several function to create arrays filled with zeros, ones, or a specific constant value.

- **Zeros Array:** `np.zeros()` function creates an array filled with zeros. It requires the shape of the array as a parameter.

Example:

```
import numpy as np
# 3x4 array filled with zeros
arr_zero = np.zeros((3, 4))
print(arr_zero)
```

Output

```
[[0. 0. 0. 0.]

 [0. 0. 0. 0.]

 [0. 0. 0. 0.]]
```

- **Ones Array:** `np.ones()` creates an array filled with ones.

Example:

```
import numpy as np
# 2x3 array filled with ones
arr_one = np.ones((2, 3))
print(arr_one)
```

Output

```
[[1. 1. 1.]
 [1. 1. 1.]]
```

- **Full Array :** `np.full()` function allows you to create an array filled with a specific value.

Example:

```
import numpy as np
# 2x2 array filled with 7
arr_full = np.full((2, 2), 7)
print(arr_full)
```

Output

```
[[7 7]
 [7 7]]
```

Creating Arrays with Random Values

NumPy also has functions for generating arrays with random values, useful for simulations and testing.

- **Random Float Array :** `np.random.rand()` function generates an array of random values between 0 and 1.

Example:

```
import numpy as np
# 2x3 array of random floats
arr_rand = np.random.rand(2, 3)
print(arr_rand)
```

Output

```
[[0.67820861 0.64484802 0.48673431]
 [0.00263043 0.55383721 0.43240166]]
```

- **Random Integers :** If we need random integers, we can use `np.random.randint()` to create arrays with integer values in a specified range.

Example:

```
import numpy as np
# 3x3 array of random integers from 1 to 9
arr_int = np.random.randint(1, 10, size=(3, 3))
print(arr_int)
```

Output

```
[[4 6 5]
 [7 4 8]
 [8 5 2]]
```

Creating Arrays with a Range of Values

Another common method of creating arrays is using a range of values. NumPy provides functions like `np.arange()` and `np.linspace()` for this purpose.

- **Using `np.arange()`** : `np.arange()` creates arrays with values spaced according to a given interval. It's similar to Python's built-in `range()` but returns a NumPy array.

Example:

```
import numpy as np
# Array from 0 to 10 with step 2
arr_range = np.arange(0, 10, 2)
print(arr_range)
```

Output

```
[0 2 4 6 8]
```

- **Using `np.linspace()`**: `np.linspace()` generates an array with a specified number of evenly spaced values over a specified range.

`np.linspace(start, stop, num)`

start = 0 → Starting value

stop = 1 → Ending value

num = 5 → Total number of values required

Example:

```
import numpy as np
# 5 values from 0 to 1
arr_linspace = np.linspace(0, 1, 5)
print(arr_linspace)
```

Output

```
[0.  0.25 0.5  0.75 1. ]
```

Identity and Diagonal Matrices

NumPy also provides functions for creating identity matrices and diagonal matrices, which are often used in linear algebra.

- **Identity Matrix** : np.eye() function creates an identity matrix, a square matrix with ones on the diagonal and zeros elsewhere.

What is an Identity Matrix?

An **identity matrix** is a square matrix in which:

- All diagonal elements are **1**
- All other elements are **0**

Example (3×3):

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Example:

```
import numpy as np
# 3x3 identity matrix
identity_matrix = np.eye(3)
print(identity_matrix)
```

Output

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

- **Diagonal Matrix** : Use `np.diag()` to create a diagonal matrix, where the provided array elements form the diagonal.

Example:

```
import numpy as np

# Diagonal matrix with [1, 2, 3] on the diagonal
diag_matrix = np.diag([1, 2, 3])
print(diag_matrix)
```

It creates a square matrix where:

- The values **1, 2, 3** are placed on the main diagonal
- All other elements are **0**

Output

```
[[1 0 0]
 [0 2 0]
 [0 0 3]]
```

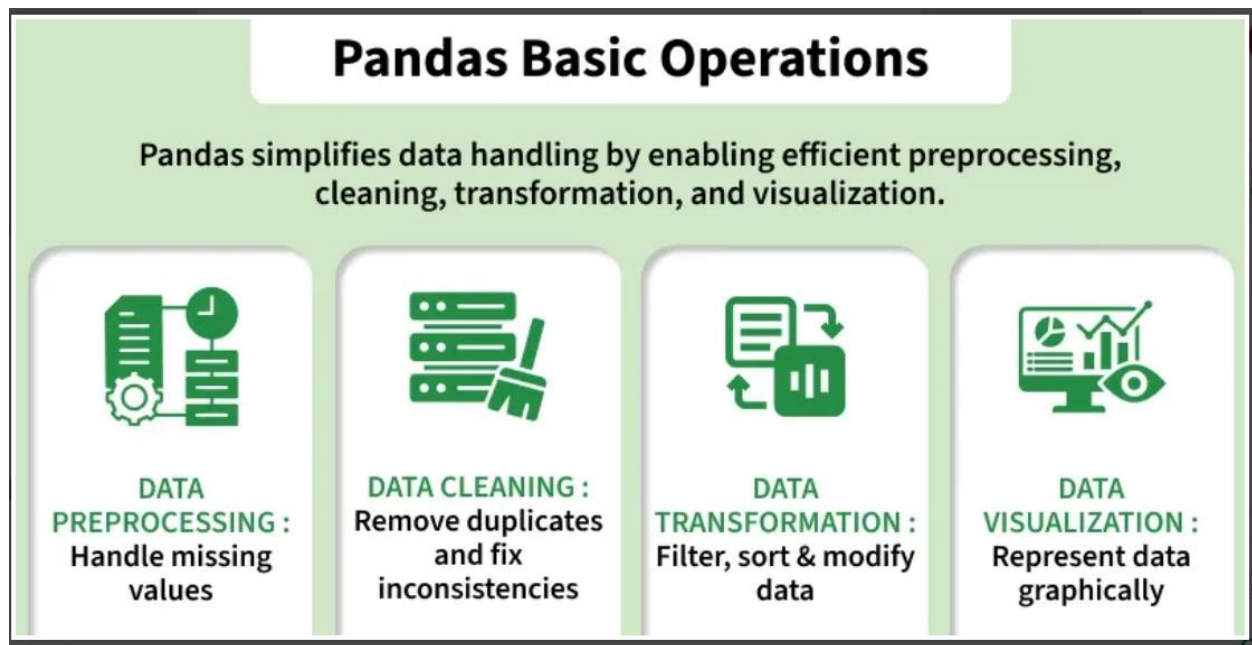
Pandas

Pandas is an open-source Python library used for data manipulation, analysis and cleaning. It provides fast and flexible tools to work with tabular data, similar to spreadsheets or SQL tables.

Pandas is used in data science, machine learning, finance, analytics and automation because it integrates smoothly with other libraries such as:

- NumPy: numerical operations
- Matplotlib and Seaborn: data visualization
- SciPy: statistical analysis
- Scikit-learn: machine learning workflows

With Pandas, you can load data, clean it, transform it, analyze it, visualize it all in just a few lines of code.



Installation

Before using Pandas, make sure it is installed:

```
pip install pandas
```

After the Pandas have been installed in the system we need to import the library.

This module is imported using:

```
import pandas as pd
```

Data Structures in Pandas

Pandas provides two data structures for manipulating data which are as follows:

1. Pandas Series

A Pandas Series is one-dimensional labeled array capable of holding data of any type (integer, string, float, Python objects etc.). The axis labels are collectively called **indexes**. Series is created by loading the datasets from existing storage which can be a SQL database, a CSV file or an Excel file.

```
import pandas as pd
import numpy as np
s = pd.Series()
print("Pandas Series: ", s)
data = np.array(['g', 'e', 'e', 'k', 's'])
s = pd.Series(data)
print("Pandas Series:\n", s)
```

Output

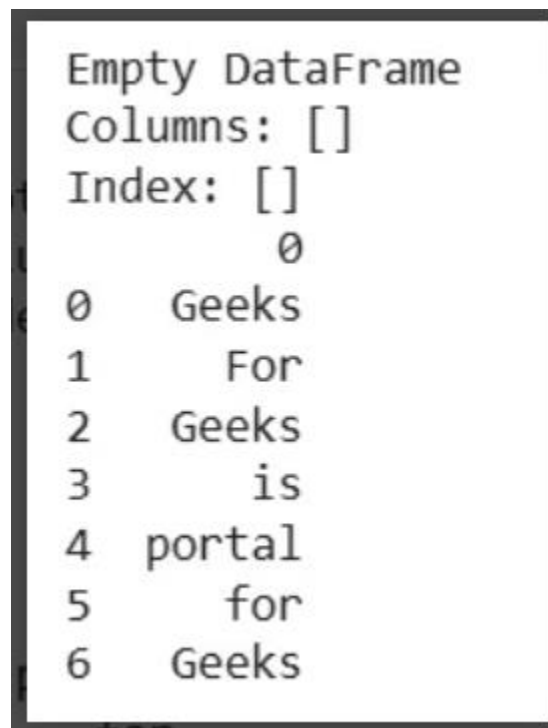
```
Pandas Series:  Series([], dtype: object)
Pandas Series:
 0    g
1    e
2    e
3    k
4    s
dtype: object
```

2. Pandas DataFrame

Pandas DataFrame is a two-dimensional data structure with labeled axes (rows and columns). It is created by loading the datasets from existing storage which can be a SQL database, a CSV file or an Excel file. It can be created from lists, dictionaries, a list of dictionaries etc.

```
import pandas as pd
df = pd.DataFrame()
print(df)
lst = ['Geeks', 'For', 'Geeks', 'is', 'portal', 'for', 'Geeks']
df = pd.DataFrame(lst)
print(df)
```

Output:



```
Empty DataFrame
Columns: []
Index: []
0
0    Geeks
1      For
2    Geeks
3      is
4  portal
5     for
6    Geeks
```

Operations in Pandas

Pandas provides essential operations for working with structured data efficiently. The sections below introduce the most commonly used functionalities with short explanations and simple examples.

1. Loading Data: This operation reads data from files such as CSV, Excel or JSON into a DataFrame.

```
import pandas as pd
df = pd.read_csv("data.csv")
print(df.head())
```

Output

	name	age	city	category	sales	a	b
0	John	25	Delhi	A	200	5	10
1	Riya	22	Mumbai	B	120	2	3
2	Aman	30	Pune	A	250	7	8
3	Neha	27	Kolkata	C	300	4	6
4	Arjun	35	Jaipur	B	200	1	9

Explanation: `pd.read_csv("data.csv")` reads the CSV file and loads it into a DataFrame and `df.head()` shows the first 5 rows of the data.

2. Viewing and Exploring Data: After loading data, it is important to understand its structure and content. This methods allow you to inspect rows, summary statistics and metadata.

```
print(df.info())
```

Output

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   name        5 non-null     object
1   age         5 non-null     int64
2   city        5 non-null     object
3   category    5 non-null     object
4   sales       5 non-null     int64
5   a           5 non-null     int64
6   b           5 non-null     int64
dtypes: int64(4), object(3)
memory usage: 412.0+ bytes
None

```

3. Handling Missing Data: Datasets often contain empty or missing values. Pandas provides functions to detect, remove or replace these values.

```
print(df.isnull().sum())
```

```
df = df.fillna(0)
```

Output

```

name      0
age       0
city      0
category  0
sales     0
a         0
b         0
dtype: int64

```

Explanation: `df.fillna(0)` replaces missing values with 0.

4. Selecting and Filtering Data: This operation retrieves specific columns, rows or records that match a condition. It allows precise extraction of required information.

```

ages = df[df['age'] > 25]
print(ages)

```

Output

	name	age	city	category	sales	a	b
2	Aman	30	Pune	A	250	7	8
3	Neha	27	Kolkata	C	300	4	6
4	Arjun	35	Jaipur	B	200	1	9

Explanation `df[df['age'] > 25]` returns rows where the "age" value is greater than 25.

5. Adding and Removing Columns: You can create new columns based on existing ones or delete unwanted columns from the DataFrame.

```
df['total'] = df['a'] + df['b']  
print(df.head())
```

Output

	name	age	city	category	sales	a	b	total
0	John	25	Delhi	A	200	5	10	15
1	Riya	22	Mumbai	B	120	2	3	5
2	Aman	30	Pune	A	250	7	8	15
3	Neha	27	Kolkata	C	300	4	6	10
4	Arjun	35	Jaipur	B	200	1	9	10

Explanation: `df['total'] = df['a'] + df['b']` creates a new column named "total".

6. Grouping Data (GroupBy): Grouping allows you to organize data into categories and compute values for each group for example, sums, counts or averages.

```
res = df.groupby('category')['sales'].sum()  
print(res)
```

Output

```
category  
A      450  
B      320  
C      300  
Name: sales, dtype: int64
```

Explanation: `df.groupby('category')` divides the dataset based on the "category" column.